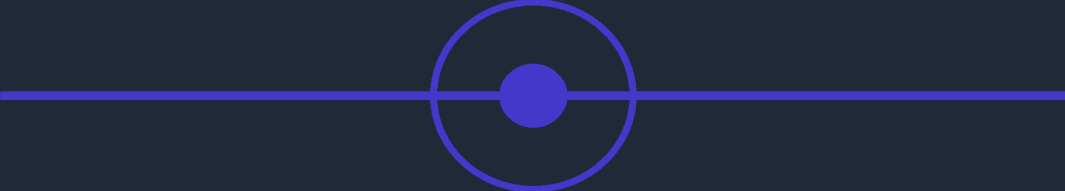


HOW I BUILT AN AUTONOMOUS AI COMPANY SOLO

*The Complete Blueprint: What Worked, What Failed, and the
Real Numbers*



OPENCLAW PRESS

What's Inside

1. The Starting Point
 2. What Actually Got Built
 3. What Failed, and Why
 4. The Real Numbers
 5. What I'd Do Differently
 6. Start Your Own
-

The Starting Point

This is a real account, not a marketing story: one founder, no team, no outside funding, using an AI coding agent as the only engineering collaborator, trying to answer a genuinely open question - can a small digital-product company actually be run with an AI agent doing the research, scoring, building, and quality-checking, with a human making the real business calls?

The company (real name withheld here by request; referred to as 'the factory' throughout) started as a single-track digital-book operation and grew, over roughly 85 recorded architecture decisions, into a multi-track system that discovers business opportunities, scores them against real evidence, and builds real product packages for the ones that clear the bar.

What this blueprint will not do: promise you a revenue number, sell you software, or claim a template you can copy and immediately profit from. What it will do: describe, honestly, what actually got built, what actually failed, and what the real numbers are as of this writing - including the fact that live checkout was still blocked by a payment provider's own onboarding process at the time this document was produced.

What Actually Got Built

The real architecture has three layers, and this separation turned out to matter more than anything else: a shared core (the scoring and evaluation logic), a set of independent product engines (one per product type), and a set of independent distribution channels (one per sales platform). When one payment platform's checkout got blocked for weeks by its own account-verification process, nothing else in the system had to change or wait - a direct, if unwelcome, proof that the separation was worth building.

A real opportunity-scoring gate sits in front of every idea: a niche has to clear a minimum profit floor and a minimum evidence-quality bar before anything gets built for it. This wasn't a one-time decision - the floor and the evidence requirements were tightened more than once after finding the gate was accepting weaker ideas than intended.

A real, automated quality gate (not a formality - it has genuinely rejected real drafts) checks every generated product for structural completeness and commercial realism before it is ever considered ready.

As of this writing: over 900 automated tests across the Python codebase and roughly 200 more in the JavaScript layer, all required to pass before any change ships. This is not a vanity number - it is the actual thing that catches a real regression before it reaches a real product.

What Failed, and Why

The most instructive failure: an early version of the opportunity-discovery pipeline evaluated 42 real candidate niches and accepted exactly zero of them. The honest investigation that followed found the real cause was not that the ideas were all bad - it was that the system matched customer-pain evidence using literal keyword search, missing real evidence written in different words. Fixing the actual root cause (teaching the system to reformulate a search query semantically before searching) mattered more than lowering the acceptance bar would have, and the acceptance bar itself was deliberately never lowered to fix this.

A second real failure, found while producing this exact document: two separate decision paths in the scoring system - a fast, single-signal gate used for most real opportunities, and a fuller, multi-signal evaluation used for deliberate review - could reach contradictory verdicts on the exact same idea. An opportunity accepted by the fast gate was re-evaluated and deferred by the fuller one, on the same day, with no code bug involved - just two real, differently-calibrated gates that had never been reconciled.

A third, more embarrassing failure: an early content-generation pipeline produced a real, paid product whose every chapter read '[Placeholder - content not yet written]' - a real product package created before the actual content-generation step had been finished, shipped without anyone noticing until it was manually reviewed months later.

A fourth: the AI content generator, even once it produced real chapter text, sometimes wrote as if the company operated a real hosted software platform with a sales team and 24/7 support - none of which existed. Confident, fluent, and factually false is a specific failure mode worth naming directly, not softening.

The Real Numbers

Honestly, as of this writing: zero completed sales. A real payment account was approved and a real product was priced and listed, but checkout itself was blocked for weeks by the payment provider's own account-verification step - a real, external, non-technical blocker, not a code problem.

The AI cost of a single real product-generation cycle: a fraction of a cent per run, measured directly from real API billing data, not estimated. The real bottleneck in this business, at least so far, has never been AI cost - it has been the non-technical steps (payment onboarding, workflow-automation platform activation) that no amount of good code can shortcut.

Roughly 85 recorded architecture decisions over the build period, each one a real, dated record of a choice made and why - a genuinely large number for a solo operation, and worth naming honestly as a real cost (time spent documenting) alongside its real benefit (nothing important was silently forgotten or re-litigated from scratch).

No fabricated metric appears anywhere in this real accounting - a section with nothing to report says so plainly rather than inventing a number, the same discipline the company's own internal rule enforces on every one of its real products.

What I'd Do Differently

Build revenue-facing infrastructure before internal governance. Multiple rounds of this build produced dashboards, reports, and internal documentation before a single real sale - valuable in hindsight, but built in the wrong order. A tighter version of this project would have chased the first real dollar much earlier and let real customer contact, not internal planning, decide what to build next.

Treat 'no fabricated data' as the single load-bearing rule, from day one. Every real quality problem found during this build traced back to some version of a shortcut that looked like real output but wasn't - placeholder text shipped as real content, an AI-written claim about software that didn't exist, a metric nobody had actually verified. The fix was never clever; it was always the same: find what's real, report what isn't, and never let the two blur.

Use the AI agent as a real collaborator with real code review, not a black box. Every meaningful bug described in this document was found by actually reading generated output - a PDF, a raw API response, a test result - not by trusting a success message. That habit, more than any specific tool choice, is what actually caught real problems before a real customer would have.

Start Your Own

Start with one narrow product type and one distribution channel, not six of each. The three-layer separation only pays off once you have more than one of something - the first one should be built simply and directly.

Resist building anything you would call 'infrastructure' or 'governance' before your first real customer interaction. This document itself is honest that its own author didn't fully follow this advice the first time.

Read the actual output your AI collaborator produces - the real file, the real API response, the real test failure - before trusting a summary of it, your own or the agent's.

This is a real account of one build, not a franchise, a course, or a guarantee. There is no ongoing support included, and no revenue number is promised or implied anywhere in this document - only what was actually true when it was written.
